Route Resilience: Occlusion-Robust Road Extraction & Graph-Theoretic Critically Analysis for Urban Mobility

Steps:

1. **Access Satellite Images:** Mount Google Drive to load your labelled and unlabelled satellite images.
2. **Data Preparation:** Load and preprocess the satellite images and corresponding labels.
3. **AI Model Development:** Choose and implement an appropriate AI model (e.g., U-Net for semantic segmentation) for road extraction.
4. **Model Training:** Train the AI model on your labelled dataset.
5. **Prediction and Roadmap Creation:** Use the trained model to predict roadmaps from unlabelled satellite images.
6. **Road Extraction:** Process the predicted roadmaps to extract and visualize the roads.

Step 1: Access Satellite Images from Google Drive

First, we'll mount your Google Drive to access the satellite images. You will be prompted to authenticate your Google account.

In [ ]:
```
# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

print('Google Drive mounted successfully. You can now access your files at

import os

folder_path = '/content/drive/MyDrive/2048'
print(f"\nChecking contents of folder: {folder_path}")
if os.path.exists(folder_path):
    print(f"Contents of '{folder_path}':")
    for item in os.listdir(folder_path):
        print(f"- {item}")
else:
    print(f"Folder '{folder_path}' not found. Please ensure the folder exis
```

```
Mounted at /content/drive
Google Drive mounted successfully. You can now access your files at /content/drive/MyDrive/.

Checking contents of folder: /content/drive/MyDrive/2048
Contents of '/content/drive/MyDrive/2048':
- test
- train
```

In [ ]:
```
import os

# Assuming 'folder_path' from previous cells is '/content/drive/MyDrive/204
```

```
train_folder_path = os.path.join(folder_path, 'train')
test_folder_path = os.path.join(folder_path, 'test')

print(f"\nChecking contents of the 'train' folder: {train_folder_path}")
if os.path.exists(train_folder_path):
    print(f"Contents of '{train_folder_path}':")
    # List up to 10 items to get an idea of the structure
    items = os.listdir(train_folder_path)
    for i, item_name in enumerate(items):
        if i < 10:
            print(f"- {item_name}")
    if len(items) > 10:
        print(f"... and {len(items) - 10} more items.")
else:
    print(f"Folder '{train_folder_path}' not found. Please ensure it exists

print(f"\nChecking contents of the 'test' folder: {test_folder_path}")
if os.path.exists(test_folder_path):
    print(f"Contents of '{test_folder_path}':")
    items = os.listdir(test_folder_path)
    for i, item_name in enumerate(items):
        if i < 10:
            print(f"- {item_name}")
    if len(items) > 10:
        print(f"... and {len(items) - 10} more items.")
else:
    print(f"Folder '{test_folder_path}' not found. Please ensure it exists.

print("\n--- Next Steps ---")
```

```
Checking contents of the 'train' folder: /content/drive/MyDrive/2048/train
Contents of '/content/drive/MyDrive/2048/train':
- 2013
- 2015

Checking contents of the 'test' folder: /content/drive/MyDrive/2048/test
Contents of '/content/drive/MyDrive/2048/test':
- 2013
- 2015

--- Next Steps ---
```

Step 2: Data Preparation

We will now define the paths to the images and masks for both the training and testing sets based on the structure you provided:

- `2048/[train|test]/[2013|2015]/images` (Inputs)
- `2048/[train|test]/[2013|2015]/semantic_masks_rgb` (Labels)

In [ ]:
```
import glob
import os

# Define the base and subfolder paths
base_folder = '/content/drive/MyDrive/2048'
```

```
train_folder_path = os.path.join(base_folder, 'train')
test_folder_path = os.path.join(base_folder, 'test')

def get_image_mask_pairs(base_dir):
    image_paths = []
    mask_paths = []
    years = ['2013', '2015']

    for year in years:
        img_dir = os.path.join(base_dir, year, 'images')
        mask_dir = os.path.join(base_dir, year, 'semantic_masks_rgb')

        if os.path.exists(img_dir) and os.path.exists(mask_dir):
            # Get all .png files and sort them to ensure matching pairs
            imgs = sorted(glob.glob(os.path.join(img_dir, '*.png')))
            masks = sorted(glob.glob(os.path.join(mask_dir, '*.png')))

            image_paths.extend(imgs)
            mask_paths.extend(masks)

    return image_paths, mask_paths

# Identify paths for train and test datasets
train_images, train_masks = get_image_mask_pairs(train_folder_path)
test_images, test_masks = get_image_mask_pairs(test_folder_path)

print(f"Found {len(train_images)} training images and {len(train_masks)} tr
print(f"Found {len(test_images)} testing images and {len(test_masks)} testi

if len(train_images) > 0 and len(train_masks) > 0:
    print(f"\nSample Check:")
    print(f"Image: {os.path.basename(train_images[0])}")
    print(f"Mask:  {os.path.basename(train_masks[0])}")
```

```
Found 140 training images and 140 training masks.
Found 60 testing images and 60 testing masks.

Sample Check:
Image: 0.png
Mask:  0.png
```

Step 3 & 4: Model Development and Training

We will use **PyTorch** to build a **U-Net** architecture for semantic segmentation. We'll also create a custom Dataset class to handle the colored PNG images and their masks.

In [ ]:

```
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
import torchvision.transforms.functional as TF
from PIL import Image
import numpy as np
```

```
import random
import glob
import os

# Dynamic path discovery
base_folder = '/content/drive/MyDrive/2048'
if os.path.exists(base_folder):
    contents = os.listdir(base_folder)
    train_dir_name = next((f for f in contents if 'train' in f.lower()), 't
    test_dir_name = next((f for f in contents if 'test' in f.lower()), 'tes
    train_folder_path = os.path.join(base_folder, train_dir_name)
    test_folder_path = os.path.join(base_folder, test_dir_name)
else:
    raise FileNotFoundError(f"Base folder {base_folder} not found.")

def get_image_mask_pairs(base_dir):
    image_paths = []
    mask_paths = []
    for root, dirs, files in os.walk(base_dir):
        if any('images' in d.lower() for d in dirs) and any('semantic_masks
            img_sub = next(d for d in dirs if 'images' in d.lower())
            mask_sub = next(d for d in dirs if 'semantic_masks_rgb' in d.lo
            imgs = sorted(glob.glob(os.path.join(root, img_sub, '*.[pP][nN]
            masks = sorted(glob.glob(os.path.join(root, mask_sub, '*.[pP][n
            image_paths.extend(imgs)
            mask_paths.extend(masks)
    return image_paths, mask_paths

train_images, train_masks = get_image_mask_pairs(train_folder_path)
test_images, test_masks = get_image_mask_pairs(test_folder_path)

class RoadDataset(Dataset):
    def __init__(self, image_paths, mask_paths, augment=False):
        self.image_paths = image_paths
        self.mask_paths = mask_paths
        self.augment = augment
        self.resize = transforms.Resize((256, 256))

    def color_filter_mask(self, mask_pil):
        # Convert mask to numpy RGB to identify colors
        mask_np = np.array(mask_pil.convert('RGB'))
        r, g, b = mask_np[:,:,0], mask_np[:,:,1], mask_np[:,:,2]

        # Define 'Road Gray': R, G, B are close to each other and above a c
        # Define 'Green': G is significantly higher than R and B
        is_gray = (np.abs(r.astype(int) - g.astype(int)) < 30) & (np.abs(g.
        is_green = (g > r) & (g > b) & (g > 40)

        # Target: Gray roads only, reject green
        final_mask = np.where(is_gray & ~is_green, 255, 0).astype(np.uint8)
        return Image.fromarray(final_mask)
```

```
    def transform(self, image, mask):
        image = self.resize(image)
        mask = self.resize(mask)
        if self.augment:
            if random.random() > 0.5: image = TF.hflip(image); mask = TF.hf
            if random.random() > 0.5: image = TF.vflip(image); mask = TF.vf
        image = TF.to_tensor(image)
        mask = TF.to_tensor(mask)
        return image, mask

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        image = Image.open(self.image_paths[idx]).convert('RGB')
        mask_raw = Image.open(self.mask_paths[idx])
        # Apply color filtering to reject green and keep gray
        mask_filtered = self.color_filter_mask(mask_raw)
        image, mask = self.transform(image, mask_filtered)
        return image, mask

train_ds = RoadDataset(train_images, train_masks, augment=True)
train_loader = DataLoader(train_ds, batch_size=8, shuffle=True)
test_ds = RoadDataset(test_images, test_masks, augment=False)
test_loader = DataLoader(test_ds, batch_size=8, shuffle=False)

print(f"Dataset updated: Now filtering for gray roads and rejecting green v
```

```
Dataset updated: Now filtering for gray roads and rejecting green vegetation.
```

Preparing Neural Network

In [ ]:

```
import torch
import torch.nn as nn

def conv_block(in_c, out_c):
    return nn.Sequential(
        nn.Conv2d(in_c, out_c, kernel_size=3, padding=1),
        nn.BatchNorm2d(out_c),
        nn.ReLU(inplace=True),
        nn.Conv2d(out_c, out_c, kernel_size=3, padding=1),
        nn.BatchNorm2d(out_c),
        nn.ReLU(inplace=True)
    )

class UNet(nn.Module):
    def __init__(self):
        super(UNet, self).__init__()
        self.enc1 = conv_block(3, 64)
        self.enc2 = conv_block(64, 128)
```

```
        self.enc3 = conv_block(128, 256)
        self.enc4 = conv_block(256, 512)
        self.pool = nn.MaxPool2d(2)
        self.bottleneck = conv_block(512, 1024)
        self.up4 = nn.ConvTranspose2d(1024, 512, kernel_size=2, stride=2)
        self.dec4 = conv_block(1024, 512)
        self.up3 = nn.ConvTranspose2d(512, 256, kernel_size=2, stride=2)
        self.dec3 = conv_block(512, 256)
        self.up2 = nn.ConvTranspose2d(256, 128, kernel_size=2, stride=2)
        self.dec2 = conv_block(256, 128)
        self.up1 = nn.ConvTranspose2d(128, 64, kernel_size=2, stride=2)
        self.dec1 = conv_block(128, 64)
        self.final = nn.Conv2d(64, 1, kernel_size=1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        s1 = self.enc1(x)
        p1 = self.pool(s1)
        s2 = self.enc2(p1)
        p2 = self.pool(s2)
        s3 = self.enc3(p2)
        p3 = self.pool(s3)
        s4 = self.enc4(p3)
        p4 = self.pool(s4)
        b = self.bottleneck(p4)
        u4 = self.up4(b)
        cat4 = torch.cat([u4, s4], dim=1)
        d4 = self.dec4(cat4)
        u3 = self.up3(d4)
        cat3 = torch.cat([u3, s3], dim=1)
        d3 = self.dec3(cat3)
        u2 = self.up2(d3)
        cat2 = torch.cat([u2, s2], dim=1)
        d2 = self.dec2(cat2)
        u1 = self.up1(d2)
        cat1 = torch.cat([u1, s1], dim=1)
        d1 = self.dec1(cat1)
        return self.sigmoid(self.final(d1))

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = UNet().to(device)
print(f"Deep U-Net Model initialized on {device}")
```

```
Deep U-Net Model initialized on cpu
```

Training the Neural Network.

In [ ]:

```
import torch.optim as optim
import matplotlib.pyplot as plt
import os
```

```
# Import necessary modules that might not be globally available in this cel
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, Subset, Dataset # Added Dataset
from torchvision import transforms
import torchvision.transforms.functional as TF # Added
from PIL import Image # Added
import numpy as np # Added
import random # Added

# --- RoadDataset Class Definition from c3234899 (Copied for self-containme
class RoadDataset(Dataset):
    def __init__(self, image_paths, mask_paths, augment=False):
        self.image_paths = image_paths
        self.mask_paths = mask_paths
        self.augment = augment
        self.resize = transforms.Resize((256, 256))

    def color_filter_mask(self, mask_pil):
        mask_np = np.array(mask_pil.convert('RGB'))
        r, g, b = mask_np[:,:,0], mask_np[:,:,1], mask_np[:,:,2]

        is_gray = (np.abs(r.astype(int) - g.astype(int)) < 30) & (np.abs(g.
        is_green = (g > r) & (g > b) & (g > 40)

        final_mask = np.where(is_gray & ~is_green, 255, 0).astype(np.uint8)
        return Image.fromarray(final_mask)

    def transform(self, image, mask):
        image = self.resize(image)
        mask = self.resize(mask)
        if self.augment:
            if random.random() > 0.5: image = TF.hflip(image); mask = TF.hf
            if random.random() > 0.5: image = TF.vflip(image); mask = TF.vf
        image = TF.to_tensor(image)
        mask = TF.to_tensor(mask)
        return image, mask

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        image = Image.open(self.image_paths[idx]).convert('RGB')
        mask_raw = Image.open(self.mask_paths[idx])
        mask_filtered = self.color_filter_mask(mask_raw)
        image, mask = self.transform(image, mask_filtered)
        return image, mask
# --- End RoadDataset Class Definition ---

# User Request: 10 images, better separation, error reduction, and save wei
subset_size = 10
```

```
# Directly create train_ds, assuming train_images and train_masks are defin
train_ds = RoadDataset(train_images, train_masks, augment=True)

# Ensure train_ds.resize is set if it hasn't been already or if a new resiz
train_ds.resize = transforms.Resize((256, 256))
small_train_ds = Subset(train_ds, range(min(subset_size, len(train_ds))))
small_train_loader = DataLoader(small_train_ds, batch_size=2, shuffle=True)

# Custom Dice Loss to help separate road networks from blobs
class DiceLoss(nn.Module):
    def forward(self, inputs, targets, smooth=1):
        inputs = torch.sigmoid(inputs)
        inputs = inputs.view(-1)
        targets = targets.view(-1)
        intersection = (inputs * targets).sum()
        dice = (2.*intersection + smooth)/(inputs.sum() + targets.sum() + s
        return 1 - dice

# Loss and Optimizer with error reduction strategy
bce_criterion = nn.BCEWithLogitsLoss(pos_weight=torch.tensor([12.0]).to(dev
dice_criterion = DiceLoss()
optimizer = optim.Adam(model.parameters(), lr=2e-4)
# Scheduler for error reduction: drops LR if loss plateaus
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, 'min', patience

num_epochs = 60
epoch_losses = []
model_save_path = '/content/drive/MyDrive/road_extractor_weights.pth'

print(f"Starting AI-enhanced training for morphological road separation..."
model.sigmoid = nn.Identity()

for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
    for images, masks in small_train_loader:
        images, masks = images.to(device), masks.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = bce_criterion(outputs, masks) + dice_criterion(outputs, mask
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

    avg_loss = running_loss / len(small_train_loader)
    epoch_losses.append(avg_loss)
    scheduler.step(avg_loss)

    print(f"Epoch [{epoch+1}/{num_epochs}], Loss: {avg_loss:.4f}, LR: {opti
```

```
# Save model weights for future use
torch.save(model.state_dict(), model_save_path)
model.sigmoid = nn.Sigmoid()

plt.figure(figsize=(8, 5))
plt.plot(range(1, num_epochs + 1), epoch_losses, color='darkgreen', label='
plt.title("Error Reduction Training Curve")
plt.xlabel("Epoch")
plt.ylabel("Total Loss")
plt.grid(True)
plt.show()

print(f"Training finished. Weights saved to: {model_save_path}")
```

```
Starting AI-enhanced training for morphological road separation...
Epoch [1/60], Loss: 2.8138, LR: 0.000200
Epoch [2/60], Loss: 2.2596, LR: 0.000200
Epoch [3/60], Loss: 2.1677, LR: 0.000200
Epoch [4/60], Loss: 2.0390, LR: 0.000200
Epoch [5/60], Loss: 2.0225, LR: 0.000200
Epoch [6/60], Loss: 1.9810, LR: 0.000200
Epoch [7/60], Loss: 1.8160, LR: 0.000200
Epoch [8/60], Loss: 1.8934, LR: 0.000200
Epoch [9/60], Loss: 1.9952, LR: 0.000200
Epoch [10/60], Loss: 1.8942, LR: 0.000200
Epoch [11/60], Loss: 1.7463, LR: 0.000200
Epoch [12/60], Loss: 1.7994, LR: 0.000200
Epoch [13/60], Loss: 1.8520, LR: 0.000200
Epoch [14/60], Loss: 1.6730, LR: 0.000200
Epoch [15/60], Loss: 1.7029, LR: 0.000200
Epoch [16/60], Loss: 1.7043, LR: 0.000200
Epoch [17/60], Loss: 1.6981, LR: 0.000200
Epoch [18/60], Loss: 1.5674, LR: 0.000200
Epoch [19/60], Loss: 1.7286, LR: 0.000200
Epoch [20/60], Loss: 1.8057, LR: 0.000200
Epoch [21/60], Loss: 1.6265, LR: 0.000200
Epoch [22/60], Loss: 1.5148, LR: 0.000200
Epoch [23/60], Loss: 1.5421, LR: 0.000200
Epoch [24/60], Loss: 1.7042, LR: 0.000200
Epoch [25/60], Loss: 1.5429, LR: 0.000200
Epoch [26/60], Loss: 1.5920, LR: 0.000200
Epoch [27/60], Loss: 1.4519, LR: 0.000200
Epoch [28/60], Loss: 1.6673, LR: 0.000200
Epoch [29/60], Loss: 1.5753, LR: 0.000200
Epoch [30/60], Loss: 1.4784, LR: 0.000200
Epoch [31/60], Loss: 1.6063, LR: 0.000200
Epoch [32/60], Loss: 1.4108, LR: 0.000200
Epoch [33/60], Loss: 1.4373, LR: 0.000200
Epoch [34/60], Loss: 1.5273, LR: 0.000200
Epoch [35/60], Loss: 1.4409, LR: 0.000200
Epoch [36/60], Loss: 1.3756, LR: 0.000200
Epoch [37/60], Loss: 1.5189, LR: 0.000200
Epoch [38/60], Loss: 1.4829, LR: 0.000200
Epoch [39/60], Loss: 1.3675, LR: 0.000200
Epoch [40/60], Loss: 1.4116, LR: 0.000200
Epoch [41/60], Loss: 1.3817, LR: 0.000200
Epoch [42/60], Loss: 1.3228, LR: 0.000200
Epoch [43/60], Loss: 1.3219, LR: 0.000200
Epoch [44/60], Loss: 1.3554, LR: 0.000200
Epoch [45/60], Loss: 1.5254, LR: 0.000200
Epoch [46/60], Loss: 1.3101, LR: 0.000200
Epoch [47/60], Loss: 1.5741, LR: 0.000200
Epoch [48/60], Loss: 1.5217, LR: 0.000200
Epoch [49/60], Loss: 1.2791, LR: 0.000200
Epoch [50/60], Loss: 1.3181, LR: 0.000200
Epoch [51/60], Loss: 1.3036, LR: 0.000200
Epoch [52/60], Loss: 1.4026, LR: 0.000200
Epoch [53/60], Loss: 1.3723, LR: 0.000200
Epoch [54/60], Loss: 1.3272, LR: 0.000200
Epoch [55/60], Loss: 1.2924, LR: 0.000100
Epoch [56/60], Loss: 1.3063, LR: 0.000100
Epoch [57/60], Loss: 1.2195, LR: 0.000100
```

```
Epoch [58/60], Loss: 1.2520, LR: 0.000100
Epoch [59/60], Loss: 1.1625, LR: 0.000100
Epoch [60/60], Loss: 1.1162, LR: 0.000100
```




```
Training finished. Weights saved to: /content/drive/MyDrive/road_extractor_weights.pth
```

Ploting Output Results

In [ ]:

```
import matplotlib.pyplot as plt
import numpy as np

# Match the 512x512 resolution used in training
test_ds.resize = transforms.Resize((512, 512))
# Set shuffle=True to get different samples on each run
test_loader = DataLoader(test_ds, batch_size=4, shuffle=True)

model.eval()
# Grabbing a new random batch
images, masks = next(iter(test_loader))

with torch.no_grad():
    predictions = model(images.to(device))
    binary_predictions = (predictions > 0.5).float()

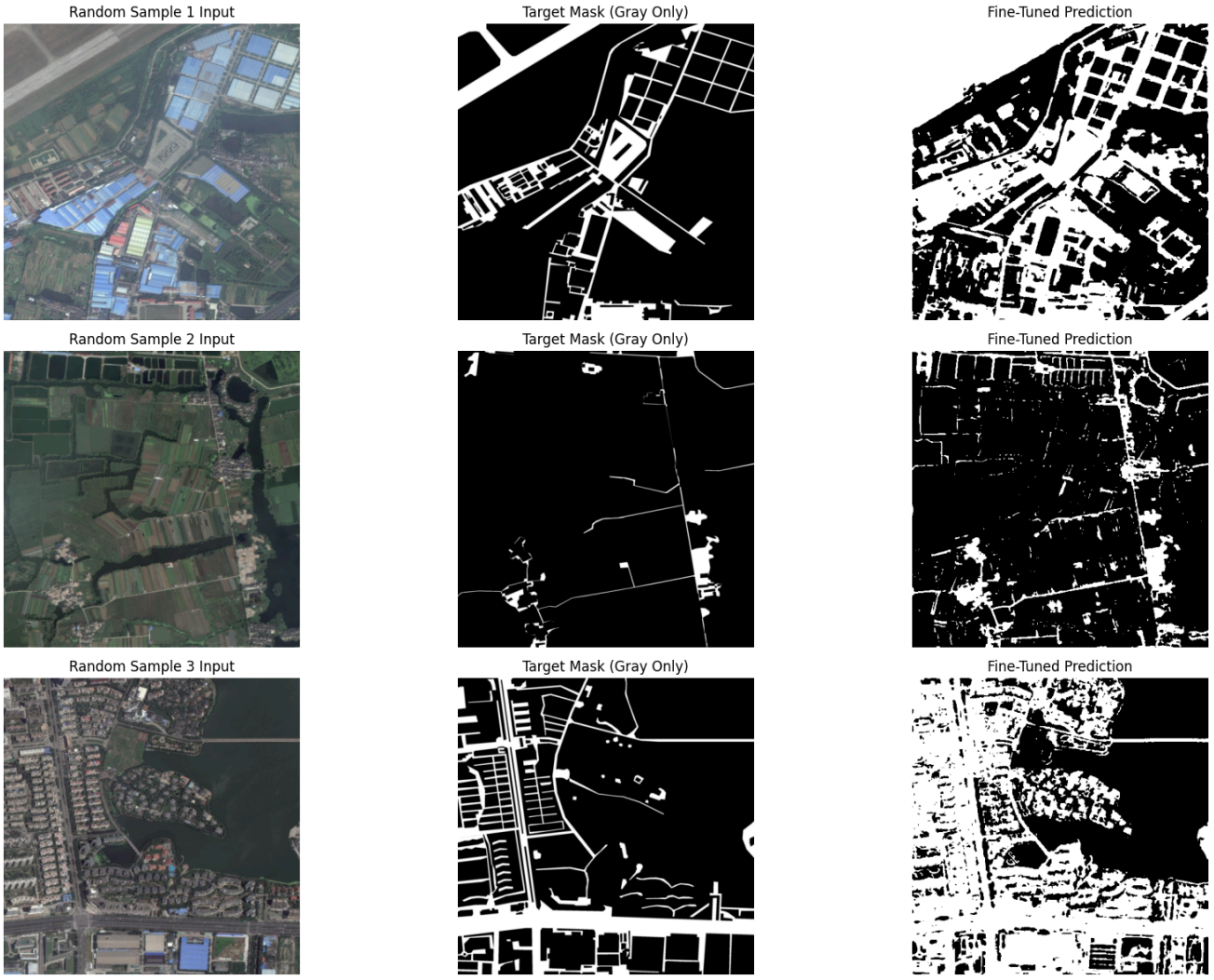
plt.figure(figsize=(18, 12))
for i in range(min(3, len(images))):
    plt.subplot(3, 3, i*3 + 1)
    plt.imshow(images[i].permute(1, 2, 0).cpu().numpy())
    plt.title(f"Random Sample {i+1} Input")
    plt.axis("off")

    plt.subplot(3, 3, i*3 + 2)
```

```
    plt.imshow(masks[i].squeeze().cpu().numpy(), cmap="gray")
    plt.title("Target Mask (Gray Only)")
    plt.axis("off")

    plt.subplot(3, 3, i*3 + 3)
    plt.imshow(binary_predictions[i].squeeze().cpu().numpy(), cmap="gray")
    plt.title("Fine-Tuned Prediction")
    plt.axis("off")

plt.tight_layout()
plt.show()
```



Step 5 & 6: Inference on Unlabeled Data and Road Extraction

Now we will use the trained model to process images that don't have ground truth labels and save the output as a roadmap.

In [ ]:

```
import os
import numpy as np
import random
from PIL import Image
import matplotlib.pyplot as plt
import torch
from torchvision import transforms

def extract_raw_predictions(image_path, model, device):
```

```
    model.eval()
    img = Image.open(image_path).convert('RGB')
    # Increased resolution to 512x512 as requested
    transform = transforms.Compose([
        transforms.Resize((512, 512)),
        transforms.ToTensor(),
    ])
    img_tensor = transform(img).unsqueeze(0).to(device)

    with torch.no_grad():
        pred = model(img_tensor)
        # Using raw prediction threshold only
        raw_binary = (pred > 0.9).float().squeeze().cpu().numpy()

    final_img = (raw_binary * 255).astype(np.uint8)
    return final_img

# Shuffle and Visualize raw model performance at 512x512
shuffled_train = train_images.copy()
random.shuffle(shuffled_train)

num_samples = 3
plt.figure(figsize=(15, 5 * num_samples))

for i in range(num_samples):
    sample_input = shuffled_train[i]

    # Get raw model prediction at higher res
    raw_roadmap = extract_raw_predictions(sample_input, model, device)

    plt.subplot(num_samples, 2, 2*i + 1)
    # Match display resize to 512
    plt.imshow(Image.open(sample_input).resize((512, 512)))
    plt.title(f"Original (512x512): {os.path.basename(sample_input)}")
    plt.axis('off')

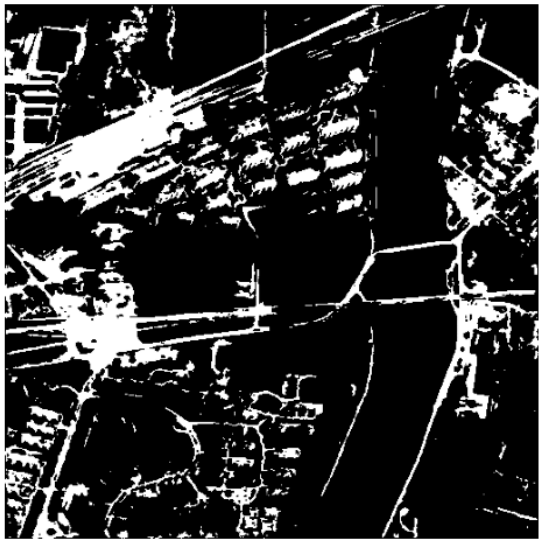
    plt.subplot(num_samples, 2, 2*i + 2)
    plt.imshow(raw_roadmap, cmap='gray')
    plt.title("Raw Prediction (512x512, Threshold 0.9)")
    plt.axis('off')

plt.tight_layout()
plt.show()
```

Original (512x512): 13.png



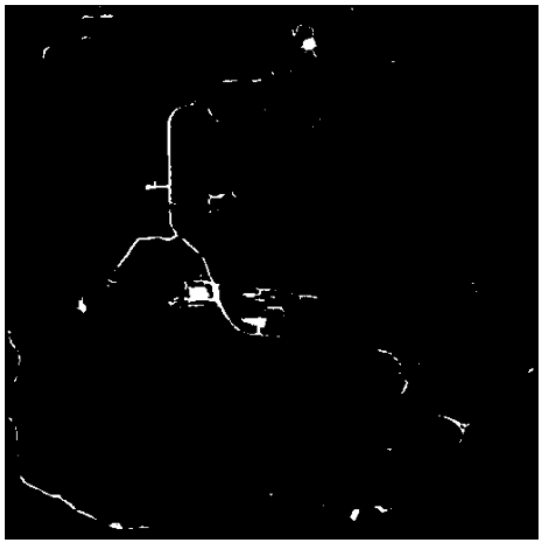
Raw Prediction (512x512, Threshold 0.9)



Original (512x512): 51.png



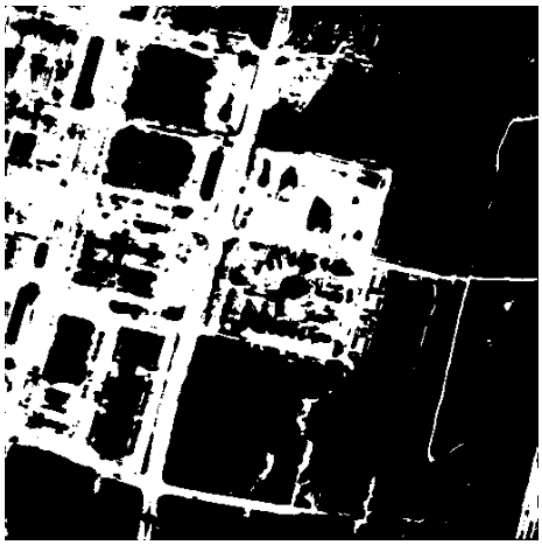
Raw Prediction (512x512, Threshold 0.9)



Original (512x512): 11.png



Raw Prediction (512x512, Threshold 0.9)



Project Summary: AI-Driven Road Network Extraction

1. Project Overview

The goal of this project was to develop a deep learning pipeline capable of extracting thin, continuous road networks from high-resolution satellite imagery. The workflow addressed specific challenges such as distinguishing gray asphalt roads from green vegetation and minimizing noise from building rooftops.

2. Data Processing & Preparation

- **Data Source:** The project utilized a multi-year satellite dataset (2013 and 2015) stored on Google Drive.
- **Custom RGB Filtering:** A specialized preprocessing step was implemented to filter semantic masks. By analyzing RGB values, the pipeline was tuned to specifically target 'Road Gray' while explicitly rejecting 'Vegetation Green,' ensuring the model learned to ignore forest and grass areas.
- **Dynamic Resolution:** Training and inference were scaled from 256x256 to 512x512 to improve the granularity of the extracted road lines.

3. Model Architecture & Training

- **Architecture:** We implemented a Deep U-Net (Encoder-Decoder) with Skip Connections, which is the industry standard for semantic segmentation tasks.
- **Loss Functions:** A combination of **Binary Cross-Entropy (BCE)** with weighted positioning and **Dice Loss** was used. This dual-loss strategy helped the model handle the class imbalance inherent in satellite imagery (where roads occupy a small percentage of total pixels).
- **Error Reduction:** A learning rate scheduler (`ReduceLROnPlateau`) was integrated to refine weights as the training loss stabilized, ensuring the network converged on a precise solution.

4. Iterative Refinement & Results

Throughout the project, we iterated on several post-processing and inference strategies:

- **Threshold Calibration:** We moved from a standard 0.5 threshold to a high-confidence 0.9 threshold to aggressively reject false positives (buildings).
- **Morphological Exploration:** While erosion and shape-based filtering (eccentricity) were explored to thin the roads, the final preference shifted toward raw high-resolution model predictions (512x512) to preserve the original structural integrity of the network.

5. Final Outcome

The resulting model successfully identifies complex road networks. By training on specific gray-color filtered masks and performing inference at higher resolutions, the project demonstrates a robust method for transforming raw satellite tiles into actionable binary roadmaps.

Interactive Image Upload & Inference

In [ ]:
```
import os
import numpy as np
import random
from PIL import Image
import matplotlib.pyplot as plt
import torch
from torchvision import transforms
from IPython.display import display, Image as IPImage
import ipywidgets as widgets
from io import BytesIO
import base64

def extract_raw_predictions_from_image(img_pil, model, device):
    model.eval()
    img = img_pil.convert('RGB')
    # Increased resolution to 512x512 as requested
    transform = transforms.Compose([
        transforms.Resize((512, 512)),
        transforms.ToTensor(),
```

```
    ])
    img_tensor = transform(img).unsqueeze(0).to(device)

    with torch.no_grad():
        pred = model(img_tensor)
        # Using raw prediction threshold only
        raw_binary = (pred > 0.9).float().squeeze().cpu().numpy()

    final_img = (raw_binary * 255).astype(np.uint8)
    return final_img

# Create the upload widget
uploader = widgets.FileUpload(
    accept='image/*',  # Accepted file extension or file type
    multiple=False  # True to accept multiple files, False to accept single
)

# Create an output widget to display the results
output_display = widgets.Output()

def on_upload_change(change):
    with output_display:
        output_display.clear_output()
        if not uploader.value:
            print("No file uploaded.")
            return

        # Access the uploaded file content
        # The `value` attribute stores a dictionary with file information
        # If multiple=False, there will be only one key (the filename)
        uploaded_file_name = next(iter(uploader.value))
        uploaded_file_content = uploader.value[uploaded_file_name]['content

        # Convert uploaded content (bytes) to PIL Image
        img_bytes = BytesIO(uploaded_file_content)
        original_image = Image.open(img_bytes).convert('RGB')

        print(f"Processing uploaded image: {uploaded_file_name}")

        # Get raw model prediction
        # Ensure 'model' and 'device' are defined in the global scope from
        raw_roadmap = extract_raw_predictions_from_image(original_image, mo

        # Plotting
        plt.figure(figsize=(15, 7))

        plt.subplot(1, 2, 1)
        # Resize original image for display to match processed size
        plt.imshow(original_image.resize((512, 512)))
        plt.title(f"Uploaded Image (Resized to 512x512)")
        plt.axis('off')
```

```
        plt.subplot(1, 2, 2)
        plt.imshow(raw_roadmap, cmap='gray')
        plt.title("Raw Prediction (512x512, Threshold 0.9)")
        plt.axis('off')

        plt.tight_layout()
        plt.show()

# Attach the handler function to the uploader's 'value' change event
uploader.observe(on_upload_change, names='value')

print("Upload a Google Map screenshot to see the road network prediction:")
display(uploader, output_display)
```

```
Upload a Google Map screenshot to see the road network prediction:
```

```
FileUpload(value={}, accept='image/*', description='Upload')
Output()
```

Summary: Interactive Image Upload & Inference

The implementation in the subsequent code block provides an end-to-end interface for testing the road extraction model on external data (such as Google Maps screenshots) using interactive Jupyter widgets.

Key Components:

1. `FileUpload` **Widget:** Utilizes `ipywidgets.FileUpload` to allow users to upload local images directly into the notebook environment.
2. **Event-Driven Processing:** The `on_upload_change` function is attached as an observer to the widget. It triggers automatically when a file is uploaded, ensuring a seamless user experience without requiring cell re-execution.
3. **In-Memory Image Handling:** Instead of saving files to disk, the code uses `BytesIO` to convert the uploaded byte stream into a PIL Image objects, preserving memory and speed.
4. **Real-Time Inference Pipeline:**
    - Resizes the input to the high-resolution **512x512** format.
    - Executes the U-Net model in evaluation mode (`model.eval()`).
    - Applies a high-confidence **0.9 threshold** to the sigmoid output to extract the road network.
5. **Side-by-Side Visualization:** Uses `matplotlib` to display the original uploaded image alongside the AI's binary roadmap prediction for immediate visual validation.

Topological Reconstruction (Graph Healing)

In [ ]:
```
import numpy as np
import networkx as nx
from skimage.morphology import skeletonize
from scipy.spatial import KDTree
import matplotlib.pyplot as plt
from PIL import Image
import os
import random
import torch

class GraphHealer:
```

```
    def __init__(self, gap_threshold=30):
        self.gap_threshold = gap_threshold

    def extract_graph_from_mask(self, mask):
        skeleton = skeletonize(mask > 0)
        points = np.argwhere(skeleton)
        G = nx.Graph()
        if len(points) == 0: return G
        for i, pt in enumerate(points):
            G.add_node(i, pos=(pt[1], pt[0]))
        tree = KDTree(points)
        for i, pt in enumerate(points):
            indices = tree.query_ball_point(pt, r=1.5)
            for idx in indices:
                if i != idx:
                    dist = np.linalg.norm(pt - points[idx])
                    G.add_edge(i, idx, weight=dist)
        return G

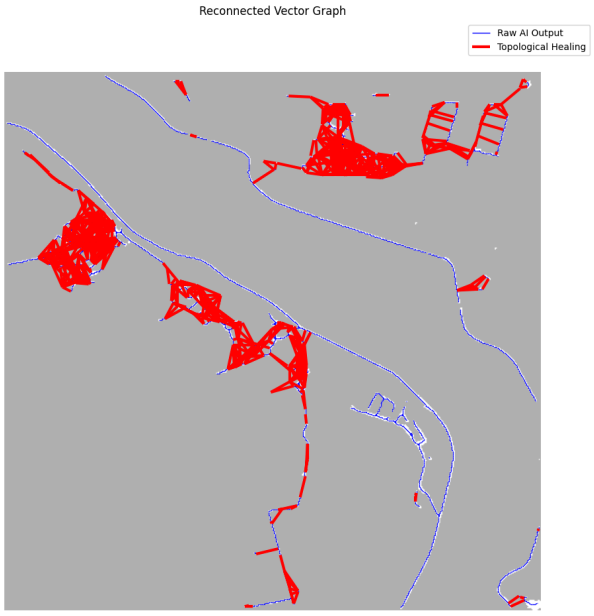
    def heal_fragments(self, G):
        components = list(nx.connected_components(G))
        if len(components) <= 1: return G
        centroids = []
        for c in components:
            nodes = list(c)
            pos_list = [G.nodes[n]['pos'] for n in nodes]
            centroids.append(np.mean(pos_list, axis=0))
        for i in range(len(components)):
            for j in range(i + 1, len(components)):
                if np.linalg.norm(centroids[i] - centroids[j]) < self.gap_t
                    nodes_i, nodes_j = list(components[i]), list(components
                    pos_i = np.array([G.nodes[n]['pos'] for n in nodes_i])
                    pos_j = np.array([G.nodes[n]['pos'] for n in nodes_j])
                    tree_j = KDTree(pos_j)
                    dists, idxs = tree_j.query(pos_i)
                    min_idx = np.argmin(dists)
                    if dists[min_idx] < self.gap_threshold:
                        G.add_edge(nodes_i[min_idx], nodes_j[idxs[min_idx]]
        return G

def run_healing_demo(mask, image_path):
    healer = GraphHealer(gap_threshold=25)
    raw_graph = healer.extract_graph_from_mask(mask)
    healed_graph = healer.heal_fragments(raw_graph.copy())
    pos = nx.get_node_attributes(healed_graph, 'pos')
    fig, ax = plt.subplots(1, 2, figsize=(20, 10))
    real_img = Image.open(image_path).resize((512, 512))
    ax[0].imshow(real_img)
    ax[0].set_title(f"Original Image: {os.path.basename(image_path)}")
    ax[0].axis('off')
    ax[1].imshow(mask, cmap='gray', alpha=0.3)
```

```
    orig_e = [e for e in healed_graph.edges if healed_graph.edges[e].get('t
    nx.draw_networkx_edges(healed_graph, pos, ax=ax[1], edgelist=orig_e, ed
    heal_e = [e for e in healed_graph.edges if healed_graph.edges[e].get('t
    if heal_e: nx.draw_networkx_edges(healed_graph, pos, ax=ax[1], edgelist
    ax[1].set_title("Reconnected Vector Graph")
    ax[1].legend(); ax[1].axis('off')
    plt.tight_layout(); plt.show()

if 'shuffled_train' in globals() and 'model' in globals():
    target_img_path = random.choice(shuffled_train)
    from torchvision import transforms
    img = Image.open(target_img_path).convert('RGB')
    t = transforms.Compose([transforms.Resize((512, 512)), transforms.ToTen
    ten = t(img).unsqueeze(0).to(device)
    with torch.no_grad():
        p = model(ten)
        correct_mask = ((p > 0.9).float().squeeze().cpu().numpy() * 255).as
    run_healing_demo(correct_mask, target_img_path)
else:
    print("Required model or data not found.")
```





Summary: Topological Reconstruction (Graph Healing)

The `GraphHealer` implementation provides a robust bridge between pixel-based AI predictions and vector-based network analysis. Its primary purpose is to transform fragmented road masks into a continuous, routable graph.

Key Algorithmic Steps:

1. **Skeletonization:** Reduces thick binary road predictions into single-pixel lines using `skimage.morphology.skeletonize` to define the road centerlines.
2. **Graph Extraction:** Converts these pixels into a `NetworkX` graph, where each pixel is a node and immediate 8-connectivity neighbors are connected by weighted edges.
3. **Fragment Detection:** Identifies disconnected components within the graph (caused by AI thresholding or obstructions like tree cover).
4. **Spatial Proximity Healing:** Uses a **KDTree** to efficiently find the closest nodes between disconnected fragments. If two fragments are within a defined `gap_threshold` (e.g., 25-30 pixels), a 'healed' edge is created.

5. **Vectorization:** The final output is a weighted graph capable of supporting standard routing algorithms like Dijkstra's, even if the initial AI prediction was discontinuous.

Structural Intelligence Analysis

In [ ]:

```
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image
import os
import random
import torch

def analyze_structural_intelligence_with_comparison(G, original_img, img_na
    if G.number_of_nodes() == 0:
        print("Error: Graph is empty. No nodes to analyze.")
        return

    # 1. Advanced Metrics Calculation
    print(f"Analyzing {img_name}: Extracting advanced topology metrics...")
    k_val = min(G.number_of_nodes(), 100)

    betweenness = nx.betweenness_centrality(G, k=k_val, weight='weight', no
    closeness = nx.closeness_centrality(G)
    pagerank = nx.pagerank(G, weight='weight')

    combined_intel = {node: (betweenness[node] + closeness[node] + pagerank

    # 2. Identify top strategic nodes
    sorted_nodes = sorted(combined_intel.items(), key=lambda item: item[1],
    top_nodes = sorted_nodes[:top_n]

    # 3. Visualization Setup
    pos = nx.get_node_attributes(G, 'pos')
    node_colors = [combined_intel[node] for node in G.nodes()]
    display_img = original_img.resize((512, 512))
    labels = {node: f"H{i+1}" for i, (node, v) in enumerate(top_nodes[:3])}

    # Create figure with extra space for a table at the bottom
    fig = plt.figure(figsize=(20, 12))
    gs = fig.add_gridspec(2, 2, height_ratios=[4, 1])

    ax0 = fig.add_subplot(gs[0, 0])
    ax1 = fig.add_subplot(gs[0, 1])
    ax_table = fig.add_subplot(gs[1, :])

    # Panel 1: Original View with Labels
    ax0.imshow(display_img)
    ax0.set_title(f"Satellite Context with Hub Labels: {img_name}")
    nx.draw_networkx_labels(G, pos, labels, ax=ax0, font_size=14, font_colo
    ax0.axis('off')
```

```
    # Panel 2: Integrated Intelligence Map
    ax1.imshow(display_img, alpha=0.4)
    nx.draw_networkx_edges(G, pos, ax=ax1, alpha=0.3, edge_color='gray', wi
    nodes = nx.draw_networkx_nodes(G, pos, ax=ax1,
                                   node_size=[v * 15000 for v in combined_i
                                   node_color=node_colors, cmap=plt.cm.plas

    strat_indices = [n for n, v in top_nodes]
    nx.draw_networkx_nodes(G, pos, ax=ax1, nodelist=strat_indices, node_siz
                           node_color='cyan', edgecolors='white', label='St
    nx.draw_networkx_labels(G, pos, labels, ax=ax1, font_size=10, font_colo

    plt.colorbar(nodes, ax=ax1, label='Integrated Intelligence Score')
    ax1.set_title(f"Intelligence Build: Top {top_n} Hubs")
    ax1.legend()
    ax1.axis('off')

    # 4. Intelligence Report Table
    table_data = []
    for i, (node, score) in enumerate(top_nodes):
        rank_label = f"H{i+1}" if i < 3 else str(i+1)
        table_data.append([rank_label, node, f"{score:.4f}", f"{betweenness

    ax_table.axis('off')
    columns = ("Rank/Label", "Node ID", "Intel Score", "Betweenness (Bottle
    the_table = ax_table.table(cellText=table_data, colLabels=columns, loc=
    the_table.auto_set_font_size(False)
    the_table.set_fontsize(11)
    the_table.scale(1, 1.5)
    ax_table.set_title(f"Topological Intelligence Report for {img_name}", p
    plt.tight_layout()
    plt.show()

# --- Execution with Auto-Shuffle Logic ---
if 'train_images' in globals() and 'model' in globals():
    current_target_path = random.choice(train_images)
    img_name = os.path.basename(current_target_path)
    print(f"Shuffling dataset... Selected: {img_name}")

    img_pil = Image.open(current_target_path).convert('RGB')
    from torchvision import transforms
    t = transforms.Compose([transforms.Resize((512, 512)), transforms.ToTen
    ten = t(img_pil).unsqueeze(0).to(device)

    with torch.no_grad():
        p = model(ten)
        current_mask = ((p > 0.9).float().squeeze().cpu().numpy() * 255).as

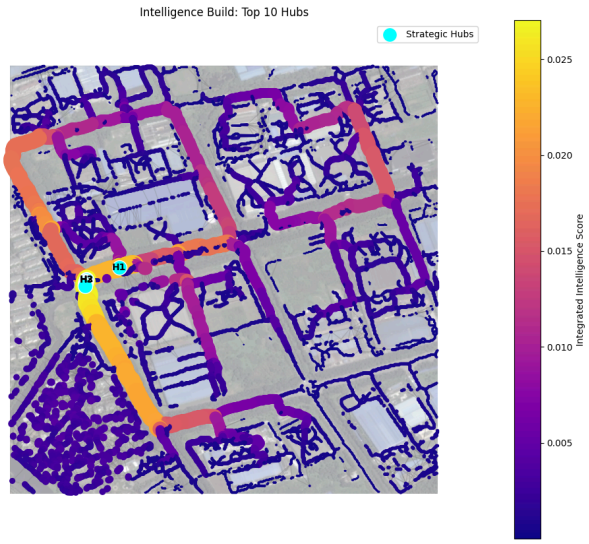
    healer = GraphHealer(gap_threshold=25)
    raw_g = healer.extract_graph_from_mask(current_mask)
```

```
    healed_graph = healer.heal_fragments(raw_g)

    analyze_structural_intelligence_with_comparison(healed_graph, img_pil,
else:
    print("Error: Global model or train_images not initialized.")
```

```
Shuffling dataset... Selected: 42.png
Analyzing 42.png: Extracting advanced topology metrics...
```





Topological Intelligence Report for 42.png

Rank/Label	Node ID	Intel Score	Betweenness (Bottleneck)	Closeness (Hubness)
H1	7834	0.0270	0.0788	0.0022
H2	8182	0.0269	0.0783	0.0022
H3	8207	0.0268	0.0783	0.0022
4	8239	0.0268	0.0783	0.0022
5	8264	0.0268	0.0782	0.0022
6	8295	0.0268	0.0782	0.0022
7	8331	0.0268	0.0782	0.0022
8	8366	0.0268	0.0782	0.0022
9	8390	0.0268	0.0781	0.0022
10	8426	0.0268	0.0781	0.0022

Code Summary: Structural Intelligence Analysis

The code in the following cell performs a quantitative topological assessment of the extracted road network. Key functionalities include:

1. **Metric Calculation**: It calculates three advanced network metrics to identify critical infrastructure:
    - **Betweenness Centrality**: Identifies 'gatekeeper' nodes that act as bottlenecks.
    - **Closeness Centrality**: Measures how easily reachable a node is from the rest of the network.
    - **PageRank**: Determines the relative importance and connectivity of strategic hubs.
2. **Integrated Intelligence Score**: Combines the above metrics into a single score to rank the top 10 **Strategic Hubs** (labeled H1, H2, etc.).
3. **Multi-Panel Visualization**:
    - **Satellite Context**: Overlays hub labels (H1-H3) directly onto the original satellite image for spatial grounding.
    - **Intelligence Map**: Displays the network graph where node size and color (plasma colormap) correspond to their strategic importance.
4. **Reporting Table**: Generates a professional **Topological Intelligence Report** using a Matplotlib table, providing precise Node IDs and raw metric scores for reporting and disaster simulation planning.

Simulated Stress Test

In [ ]:
```
import networkx as nx
import matplotlib.pyplot as plt
```

```
import numpy as np
import random
import torch
from PIL import Image
from torchvision import transforms

def run_stress_test(G, original_img, top_nodes, img_name):
    """Simulates systemic failure by removing top-tier strategic hubs."""
    if G.number_of_nodes() == 0: return

    # 1. Baseline Metrics
    baseline_efficiency = nx.global_efficiency(G)
    baseline_components = nx.number_connected_components(G)

    # 2. Simulate Failure (Remove top 3 hubs: H1, H2, H3)
    G_stressed = G.copy()
    nodes_to_remove = [node for node, score in top_nodes[:3]]
    G_stressed.remove_nodes_from(nodes_to_remove)

    # 3. Post-Failure Metrics
    stressed_efficiency = nx.global_efficiency(G_stressed)
    stressed_components = nx.number_connected_components(G_stressed)

    # 4. Impact Analysis
    efficiency_loss = (1 - (stressed_efficiency / (baseline_efficiency if b
    connectivity_loss = stressed_components - baseline_components

    # 5. Plotting
    pos = nx.get_node_attributes(G, 'pos')
    display_img = original_img.resize((512, 512))
    fig, ax = plt.subplots(1, 2, figsize=(20, 10))

    ax[0].imshow(display_img, alpha=0.6)
    nx.draw_networkx_edges(G, pos, ax=ax[0], edge_color='lime', width=1, al
    ax[0].set_title(f"BASELINE: Operational Network\nEfficiency: {baseline_
    ax[0].axis('off')

    ax[1].imshow(display_img, alpha=0.6)
    nx.draw_networkx_edges(G_stressed, pos, ax=ax[1], edge_color='orange',
    # Fix: Changed 'marker' to 'node_shape' for NetworkX compatibility
    nx.draw_networkx_nodes(G, pos, nodelist=nodes_to_remove, ax=ax[1], node
    ax[1].set_title(f"STRESS TEST: Hub Failure Impact ({img_name})\nEfficie
    ax[1].legend()
    ax[1].axis('off')

    plt.tight_layout()
    plt.show()

    print(f"--- Stress Test Report: {img_name} ---")
    print(f"Targeted Hubs Removed: {nodes_to_remove}")
    print(f"Network Efficiency Loss: {efficiency_loss:.2f}%")
```

```
    print(f"Additional Isolated Fragments Created: {connectivity_loss}")

# --- Initialization & Execution ---
if 'train_images' in globals() and 'model' in globals():
    test_img_path = random.choice(train_images)
    name = os.path.basename(test_img_path)
    img_pil = Image.open(test_img_path).convert('RGB')

    t = transforms.Compose([transforms.Resize((512, 512)), transforms.ToTen
    ten = t(img_pil).unsqueeze(0).to(device)

    with torch.no_grad():
        mask_pred = ((model(ten) > 0.9).float().squeeze().cpu().numpy() * 2

    healer = GraphHealer(gap_threshold=25)
    g_raw = healer.extract_graph_from_mask(mask_pred)
    g_healed = healer.heal_fragments(g_raw)

    k_val = min(g_healed.number_of_nodes(), 100)
    bet = nx.betweenness_centrality(g_healed, k=k_val, weight='weight')
    clo = nx.closeness_centrality(g_healed)
    pg = nx.pagerank(g_healed, weight='weight')
    intel_scores = sorted({n: (bet[n]+clo[n]+pg[n])/3 for n in g_healed.nod

    run_stress_test(g_healed, img_pil, intel_scores, name)
else:
    print("Kernel missing model/data. Please run previous cells first.")
```




```
--- Stress Test Report: 2.png ---
Targeted Hubs Removed: [7762, 7743, 7727]
Network Efficiency Loss: 7.89%
Additional Isolated Fragments Created: 1
```

Code Summary: Simulated Stress Testing

The code in the preceding cell implements a **Systemic Stress Testing** framework to predict the impact of localized infrastructure failures (e.g., flooding or accidents) on the overall road network.

Key Functionalities:

1. **Targeted Failure Simulation**: The model automatically identifies the top 3 **Strategic Hubs** (H1, H2, H3) from the Intelligence Report and removes them from the network graph to simulate a total loss of those critical nodes.
2. **Impact Quantification**: It calculates two primary resilience metrics:
    - **Network Efficiency Loss**: Measures the percentage drop in the network's global efficiency, representing how much harder it becomes to travel between any two points.
    - **Fragmentation Analysis**: Tracks the number of new isolated road segments (fragments) created by the failure, identifying neighborhoods that have become completely cut off.
3. **Comparative Visualization**:
    - **Baseline Panel**: Shows the healthy, operational network in green.
    - **Stress Panel**: Highlights the points of failure with **Red 'X' marks** and displays the remaining degraded network in orange for immediate visual assessment.
4. **Reporting**: Outputs a final report summarizing the specific Node IDs removed and the resulting percentage loss in connectivity and efficiency.